

Custom Payload Encoder & Obfuscation Framework

Student Name : Sakshi Maruti Nalawade .

Institute : Unified Mentor

Lab Environment : Kali Linux

INDEX	
	Title
1.	Objective
2.	Tools
3.	Environment
4.	Methodology
5.	Results and Observations

Project Objectives

- **Design and develop a payload encoding framework** that supports Base64, XOR, and ROT13 encoding techniques for educational and research purposes.
- **Implement reversible string obfuscation methods** to demonstrate how data transformations can affect simple pattern-based detection.
- **Simulate a basic static detection mechanism** using keyword or signature matching to evaluate how encoded or obfuscated test inputs are identified.
- **Compare original and transformed payloads** to analyse differences in detection outcomes, readability, and structure.
- **Evaluate the effectiveness of various encoding and obfuscation techniques** against simulated signature-based detection in a controlled laboratory environment.
- **Generate detailed reports** showing encoded and obfuscated outputs, detection results, and comparative analysis to support cyber security learning and research.
- **Demonstrate the limitations of basic signature-based detection** and highlight the importance of layered and behaviour-based defensive approaches.
- **Provide a modular and extensible framework** that can be enhanced with additional encoding methods, detection rules, or analysis features for future academic work.

Tools & Technologies Used

Programming Languages:

- Python (recommended)
- Python Libraries
 1. Base64 Library
 2. Random Library
 3. Itertools Library
 4. Argparse Library (Optional)

Documentation Tools:

- Word / Google Docs
- word (flowcharts and architecture diagrams)

Phase 1: Project Planning

Step 1: Define the Goal

Build a framework that:

- Encodes data using Base64, XOR, and ROT13
- Applies string obfuscation techniques
- Simulates signature-based detection
- Compares detection results before and after obfuscation
- Generates reports showing effectiveness

Expected Output

Module	Function
Encoder	Base64, XOR, ROT13
Obfuscator	String transformation
Detection Engine	Simulated signature matching
Report Engine	Results & analysis

Phase 2: Environment Setup

Step 2: Install Python3

Check Python:

python3 --version

```
(sakshi@kali)-[~]  
$ python3 --version  
Python 3.13.3
```

python3

```
(sakshi@kali)-[~]  
$ python3  
Python 3.13.3 (main, Apr 10 2025, 21:38:51) [GCC 14.2.0] on linux  
Type "help", "copyright", "credits" or "license" for more information.  
>>> import base64  
>>>
```

Phase 3: Encoder Module

Step 3: Implement Base64 Encoding

Python Libraries:

1. Base64 Library

Used for Base64 encoding and decoding operations.

Convert plaintext into Base64 format.

```
>>>
>>> import base64
...
... data = "Hello ! Cyber Security Students"
...
... encoded = base64.b64encode(data.encode())
... print(encoded.decode())
...
SGVsbG8gISBDeWJlc iBTZWN1cm l0eSBTdHVkZW50cw==
>>>
```

```
>>> import base64
...
... data = "SGVsbG8gISBDeWJlc iBTZWN1cm l0eSBTdHVkZW50cw=="
...
... decoded = base64.b64decode(data)
...
... print(decoded.decode())
...
Hello ! Cyber Security Students
>>>
```

Functions

encode_base64()

decode_base64()

2. Random Library

Used to generate random characters for string obfuscation.

```
>>>
>>> import random
>>> import string
>>>
>>> char = random.choice (string.ascii_letters)
>>> print (char)
y
>>>
>>>
```

Use Cases:

- Random character insertion
- Payload mutation simulation
- Obfuscation testing

3. Itertools Library

Provides efficient iteration tools for string manipulation.

```
>>> import itertools
>>> parts = [ "Cyber", "Security", "Project" ]
>>> for item in itertools.chain(parts) :
...     print (item)
Cyber
Security
Project
>>>
```

Use Cases:

- Character splitting
- Combining payload fragments
- Multi-layer transformations

4. Argparse Library (Optional)

```
>>> import argparse
>>> parser = argparse.ArgumentParser ()
>>> parser.add_argument ("--encode")
_StoreAction(option_strings=['--encode'], dest='encode', nargs=None, const=None, default=None, type=None, choices=None, required=False, help=None, metavar=None, deprecated=False)
>>> args = parser.parse_args ()
>>> print (args.encode)
None
>>>
```

Benefits:

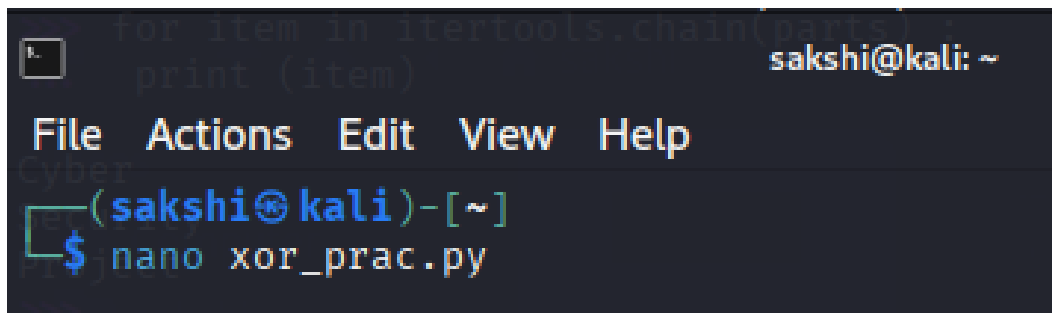
- User-friendly interface
- Supports command-line options
- Useful for demonstrations

Step 4: Implement XOR Encoding

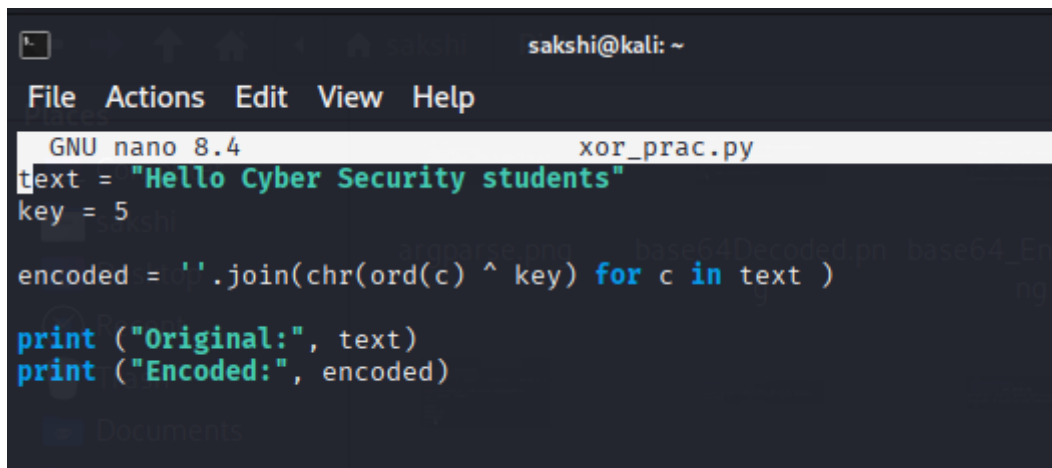
XOR:

Encode text using a user-defined key.

nano xor_prac.py

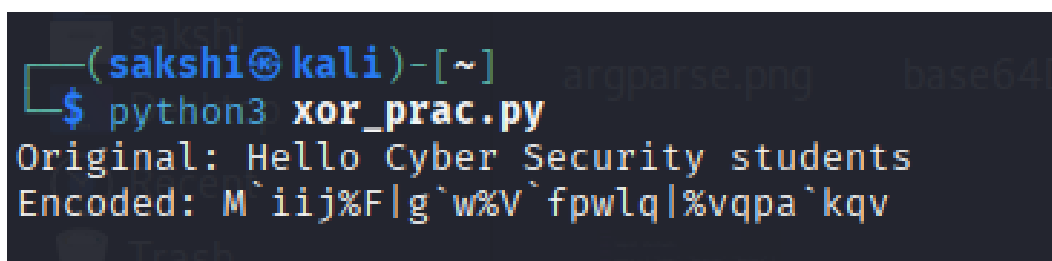


```
sakshi@kali: ~  
File Actions Edit View Help  
(sakshi@kali)-[~]  
$ nano xor_prac.py
```



```
sakshi@kali: ~  
File Actions Edit View Help  
GNU nano 8.4 xor_prac.py  
text = "Hello Cyber Security students"  
key = 5  
encoded = ''.join(chr(ord(c) ^ key) for c in text )  
print ("Original:", text)  
print ("Encoded:", encoded)
```

python3 xor_prac.py



```
(sakshi@kali)-[~]  
$ python3 xor_prac.py  
Original: Hello Cyber Security students  
Encoded: M`iij%F|g`w%V`fpwlq|%vqpa`kqv
```

Functions

xor_encode()

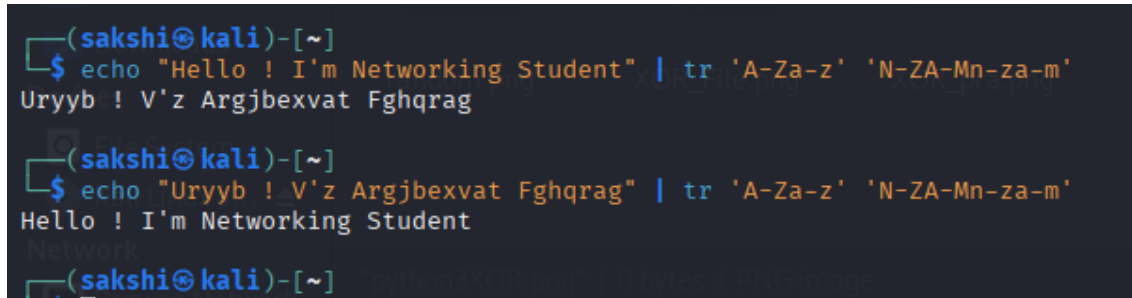
xor_decode()

Step 5: Implement ROT13 Encoding

ROT13 Encoding:

Rotate alphabet characters by 13 positions.

```
echo "Hello ! I'm Networking Student" | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```



```
(sakshi@kali)-[~]  
$ echo "Hello ! I'm Networking Student" | tr 'A-Za-z' 'N-ZA-Mn-za-m'  
Uryyb ! V'z Argjbexvat Fghqrag  
  
(sakshi@kali)-[~]  
$ echo "Uryyb ! V'z Argjbexvat Fghqrag" | tr 'A-Za-z' 'N-ZA-Mn-za-m'  
Hello ! I'm Networking Student  
Network  
(sakshi@kali)-[~]
```

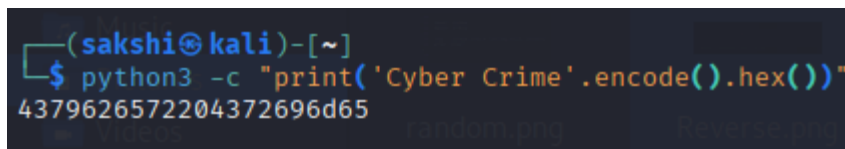
Functions

rot13_encode()

Phase 4: String Obfuscation Module

Step 6: Escape Sequence Obfuscation

```
Python3 -c "print ('Cyber Crime'.encode().hex())"
```

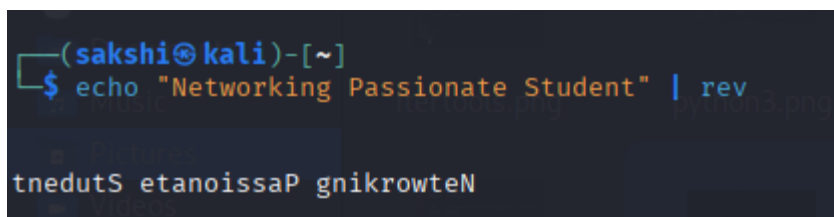


```
(sakshi@kali)-[~]  
$ python3 -c "print('Cyber Crime'.encode().hex())"  
4379626572204372696d65
```

Demonstrates alternate data representation.

Step 7: Reverse Transformation

```
echo "Networking Passionate Student" | rev
```



```
(sakshi@kali)-[~]  
$ echo "Networking Passionate Student" | rev  
tnedutS etanoissaP gnikrowteN
```

Simple but useful for testing.

Phase 5: Reporting Engine

Step 8: Create Result Table

Technique	Detected
Original	Yes
Base64	No
XOR	No
ROT13	No
ROT13 + Base64	No
Multi-Layer Obfuscation	No

Phase 6: Architecture Diagram

